DBMS Practical Exam

Exp 3 - AIM: Create a database using Data Definition Language (DDL) and apply integrity constraints for the specified System

Syntax-

```
-- Create a table with constraints
CREATE TABLE table_name (
    column1 datatype(size) PRIMARY KEY,
    column2 datatype(size) NOT NULL,
    column3 datatype(size) UNIQUE,
    column4 datatype(size) DEFAULT 'default_value',
    column5 datatype(size),
    CHECK (column5 > 18),
    FOREIGN KEY (column_name) REFERENCES other_table(column_name)
);
-- Add a new column
ALTER TABLE table_name ADD column_name datatype(size);
-- Modify column datatype (MySQL standard)
ALTER TABLE table_name MODIFY column_name new_datatype(size);
-- Rename a column (MySQL standard)
ALTER TABLE table_name RENAME COLUMN old_name TO new_name;
-- Delete all data but keep the table structure
TRUNCATE TABLE table_name;
-- Delete the table entirely
DROP TABLE table_name;
-- Rename a table
RENAME TABLE old_table_name TO new_table_name;
```

Exp 4 -AIM: To study and implement DML commands in SQL for inserting, retrieving, updating, and deleting data from database tables.

Syntax-

-- Insert a new record

```
INSERT INTO table_name (column1, column2, column3)
VALUES (value1, 'string_value2', value3);
```

-- Insert a record without specifying columns (must match exact table order)

```
INSERT INTO table_name VALUES (value1, 'value2', value3);
```

-- Retrieve all data

```
SELECT * FROM table_name;
```

-- Retrieve specific columns with a condition

```
SELECT column1, column2 FROM table_name WHERE condition = 'value';
```

-- Update existing data

```
UPDATE table_name
SET column1 = new_value, column2 = 'new_string'
WHERE condition_column = specific_value;
```

-- Delete specific data

```
DELETE FROM table_name WHERE condition_column = specific_value;
```

Exp 5 -Aim: To understand and perform Simple SQL queries, String manipulation functions and Aggregate functions using a relational database.

Syntax –

-- Retrieve all columns and rows from a table

```
SELECT * FROM EMPLOYEE;
```

-- Retrieve specific columns based on a condition

```
SELECT emp_name, salary FROM EMPLOYEE WHERE department = 'IT';
```

-- Retrieve data and sort it in descending order

```
SELECT emp_name, salary FROM EMPLOYEE ORDER BY salary DESC;
```

-- Change text case

```
SELECT UPPER(emp_name) FROM EMPLOYEE;       -- Converts to UPPERCASE
SELECT LOWER(city) FROM EMPLOYEE;           -- Converts to lowercase
```

-- Find the length of a string

```
SELECT emp_name, LENGTH(emp_name) FROM EMPLOYEE;
```

-- Extract a specific part of a string (Start at position 1, take 3 characters)

```
SELECT SUBSTRING(emp_name, 1, 3) FROM EMPLOYEE;
```

-- Join two or more strings together

```
SELECT CONCAT(emp_name, '-', department) FROM EMPLOYEE;
```

-- Replace specific text with new text

```
SELECT REPLACE(city, 'Mumbai', 'Bombay') FROM EMPLOYEE;
```

-- Reverse the text

```
SELECT REVERSE(emp_name) FROM EMPLOYEE;
```

-- Remove leading and/trailing spaces

```
SELECT TRIM(emp_name) FROM EMPLOYEE;
```

-- Count the total number of rows

```
SELECT COUNT(*) FROM EMPLOYEE;
```

-- Calculate the sum total of a numeric column

```
SELECT SUM(salary) FROM EMPLOYEE;
```

-- Calculate the average of a numeric column

```
SELECT AVG(salary) FROM EMPLOYEE;
```

-- Find the highest and lowest values in a column

```
SELECT MAX(salary), MIN(salary) FROM EMPLOYEE;
```

-- GROUP BY: Count employees per department

```
SELECT department, COUNT(*)
FROM EMPLOYEE
GROUP BY department;
```

-- HAVING: Filter grouped data (Departments with avg salary > 50000)

```
SELECT department, AVG(salary)
FROM EMPLOYEE
GROUP BY department
HAVING AVG(salary) > 50000;
```

Exp 6-Aim: To study and implement different types of JOIN operations in SQL using multiple tables and understand how data is retrieved based on relationships between tables.

Syntax-

-- 1. INNER JOIN: Returns ONLY records that have matching values in both tables

```
SELECT E.Name, D.DeptName

FROM Employees E

INNER JOIN Departments D ON E.DeptID = D.DeptID;
```

-- 2. LEFT JOIN: Returns ALL records from the left table, and matched records from the right

```
SELECT E.Name, D.DeptName

FROM Employees E

LEFT JOIN Departments D ON E.DeptID = D.DeptID;
```

-- 3. RIGHT JOIN: Returns ALL records from the right table, and matched records from the left

```
SELECT E.Name, D.DeptName

FROM Employees E

RIGHT JOIN Departments D ON E.DeptID = D.DeptID;
```

-- 4. FULL OUTER JOIN: Returns all records when there is a match in either left or right table

-- Note: MySQL uses UNION to achieve a full outer join

```
SELECT E.Name, D.DeptName

FROM Employees E LEFT JOIN Departments D ON E.DeptID = D.DeptID

UNION

SELECT E.Name, D.DeptName

FROM Employees E RIGHT JOIN Departments D ON E.DeptID = D.DeptID;
```

-- 5. LEFT JOIN EXCLUDING INNER: Find employees who are NOT assigned to any department

```
SELECT E.Name

FROM Employees E

LEFT JOIN Departments D ON E.DeptID = D.DeptID

WHERE D.DeptID IS NULL;
```

-- 6. RIGHT JOIN EXCLUDING INNER: Find departments that have NO employees assigned

```
SELECT D.DeptName

FROM Employees E

RIGHT JOIN Departments D ON E.DeptID = D.DeptID

WHERE E.DeptID IS NULL;
```

-- 7. FULL OUTER JOIN EXCLUDING INNER: Find unassigned employees AND empty departments

```
SELECT E.Name, D.DeptName

FROM Employees E LEFT JOIN Departments D ON E.DeptID = D.DeptID WHERE D.DeptID IS NULL

UNION

SELECT E.Name, D.DeptName

FROM Employees E RIGHT JOIN Departments D ON E.DeptID = D.DeptID WHERE E.DeptID IS NULL;
```

Exp 7- Aim: To study and perform Nested and Complex queries on the given schema for a specified system.

Syntax-

```
-- Using a subquery with comparison (Inner query returns a single value)
SELECT column_name FROM table_name
WHERE numeric_column > (SELECT AVG(numeric_column) FROM table_name);

-- Using a subquery with IN (Inner query returns a list of values)
SELECT column_name FROM table1
WHERE id_column IN (
   SELECT id_column FROM table2 WHERE condition = 'value'
);

-- Complex Query (Joining tables and using Group By/Having)
SELECT c.grade, COUNT(s.id) AS total_students
FROM Classes c
JOIN Students s ON c.id = s.class_id
GROUP BY c.grade
HAVING COUNT(s.id) > 1;

-- Join tables to display related information (Student name and their grade)
SELECT s.name, c.grade
FROM Students s
JOIN Classes c ON s.class_id = c.id;

-- Join multiple tables (Student, Class, and Teacher)
SELECT s.name AS Student, t.name AS Teacher
FROM Students s
```

```
JOIN Classes c ON s.class_id = c.id
JOIN Teachers t ON c.teacher_id = t.id;

-- Use GROUP BY and HAVING to filter grouped data (Grades with > 1 student)
SELECT c.grade, COUNT(s.id) AS total_students
FROM Classes c
JOIN Students s ON c.id = s.class_id
GROUP BY c.grade
HAVING COUNT(s.id) > 1;
```

Exp 8-Aim: To study and implement Data Control Language (DCL) and Transaction Control Language (TCL) commands in SQL.

Syntax-

-- Give specific privileges to a user

GRANT SELECT, INSERT ON Employee TO user1;

-- Take those privileges away

REVOKE INSERT ON Employee FROM user1;

-- 1. Start modifying data

UPDATE Employee SET salary = 40000 WHERE emp_id = 1;

-- 2. Create a temporary checkpoint

SAVEPOINT sp1;

-- 3. Make another modification

UPDATE Employee SET salary = 45000 WHERE emp_id = 2;

-- 4. Revert back to the checkpoint (Undoes the 45000 salary update, keeps the 40000 update)

ROLLBACK TO sp1;

-- 5. Save all remaining changes permanently

COMMIT;

-- Note: Using just 'ROLLBACK;' will undo ALL changes since the last COMMIT.

Exp 9-AIM: To implement SQL Views and Triggers in a relational database to perform data abstraction, enforce business rules, and automate database operations.

Syntax-

```
-- Create the View (Virtual Table)

CREATE VIEW Current_Product_List AS

SELECT ProductID, ProductName

FROM Products

WHERE Discontinued = 'No';


-- Query the view exactly like a normal table

SELECT * FROM Current_Product_List;


-- Drop the trigger if making modifications to it

DROP TRIGGER IF EXISTS price_history_trigger;


-- Create an AFTER or BEFORE Trigger

-- (This example logs the old unit price BEFORE it gets updated)

DELIMITER $$

CREATE TRIGGER price_history_trigger

BEFORE UPDATE ON product

FOR EACH ROW

BEGIN

   INSERT INTO product_price_history (product_id, product_name, supplier_name,
unit_price)

   VALUES (OLD.product_id, OLD.product_name, OLD.supplier_name, OLD.unit_price);

END$$

DELIMITER ;
```

Exp 10-AIM: To study and demonstrate transaction processing and concurrency control using locking techniques in a database system.

Syntax-

```
-- Create the accounts table for the transaction test
CREATE TABLE accounts (
   acc_no INT PRIMARY KEY,
   name VARCHAR(30),
   balance INT
);

-- Insert sample data
INSERT INTO accounts VALUES
(101, 'Amit', 5000),
(102, 'Neha', 7000);

-- 1. Start the transaction
START TRANSACTION;

-- 2. Apply an Exclusive Lock to a specific row
-- (Other transactions will now wait if they try to access account 101)
SELECT * FROM accounts
WHERE acc_no = 101
FOR UPDATE;

-- 3. Perform your update on the locked row
UPDATE accounts
SET balance = balance - 1000
WHERE acc_no = 101;
-- 4. Commit the transaction to save changes and release the lock
COMMIT;
```

Exp 10 Practice Queries

Create Table

```
CREATE TABLE accounts (
id INT PRIMARY KEY,
name VARCHAR(50),
balance INT
);
```

Insert Data

```
INSERT INTO accounts VALUES
(1, 'Amit', 1000),
(2, 'Ravi', 2000);
```

Window 1 (Transaction 1)

```
START TRANSACTION;

SELECT * FROM accounts WHERE id = 1 FOR UPDATE;  //Exclusive lock

UPDATE accounts SET balance = balance - 200 WHERE id = 1;
```

Window 2 (Transaction 2)

```
START TRANSACTION;

SELECT * FROM accounts WHERE id = 1 FOR UPDATE; //This will WAIT (BLOCKED)
Because Window 1 holds the lock
```

Now go back to Window 1

```
COMMIT;
```

Now Window 2 continues automatically Preventing Dirty Read

Window 1

```
START TRANSACTION;

UPDATE accounts SET balance = 5000 WHERE id = 1;
```

Window 2

```
SELECT * FROM accounts WHERE id = 1;
```

DELETE with Lock

Window1

```
START TRANSACTION;

SELECT * FROM accounts WHERE id = 1 FOR UPDATE;
```

DELETE FROM accounts WHERE id = 1;

Window 2

SELECT * FROM accounts WHERE id = 1; //Blocked or waits

Shared Lock (S-Lock)

Window 1

START TRANSACTION;

SELECT * FROM accounts LOCK IN SHARE MODE;

Window 2

START TRANSACTION;

SELECT * FROM accounts LOCK IN SHARE MODE;

But if someone tries UPDATE

Window 2

UPDATE accounts SET balance = 7000 WHERE id = 2; //Blocked

Reading While Preventing Write

Window 1

START TRANSACTION;

SELECT balance FROM accounts WHERE id = 2 LOCK IN SHARE MODE;

Window 2

UPDATE accounts SET balance = balance + 500 WHERE id = 2; //WAIT until Window 1 commits

Window1

COMMIT;